

Normalización z-score en ensamblador x86-64: comparación entre una implementación escalar y una vectorial AVX2

Josué Campos Herrera
Escuela de Ingeniería Electrónica
Tecnológico de Costa Rica
Cartago, Costa Rica

Eduardo Franco Román Vargas
Escuela de Ingeniería Electrónica
Tecnológico de Costa Rica
Cartago, Costa Rica

Resumen—Se presenta la implementación y evaluación de un kernel de normalización estadística (z-score) escrito en lenguaje ensamblador x86-64 con sintaxis NASM, en dos versiones funcionalmente equivalentes: una escalar, que procesa un valor por instrucción con las operaciones escalares de SSE sobre registros XMM, y una vectorial basada en AVX2 sobre registros YMM de 256 bits, que procesa ocho valores de simple precisión por iteración. Se documentan las decisiones de diseño de la versión vectorial, en particular las reducciones horizontales, el tratamiento del remanente y el manejo de casos borde. La correctitud de ambos kernels se verificó contra una referencia en doble precisión y mediante GDB sobre un caso reproducible de $N = 16$. El desempeño se midió sobre cuatro tamaños entre 10^3 y 5×10^7 elementos con treinta repeticiones por caso. La aceleración supera el factor teórico de 8x mientras los datos caben en caché, con un máximo de 11,8x, y decae a 6,4x cuando el arreglo debe leerse desde memoria principal, lo que evidencia un régimen limitado por el ancho de banda. Adicionalmente, se identificó que la acumulación secuencial en float32 pierde precisión para $N \geq 10^6$, efecto que la versión vectorial atenúa al repartir la suma en ocho acumuladores.

Index Terms—SIMD, AVX2, ensamblador x86-64, NASM, vectorización, z-score, ancho de banda de memoria, precisión de punto flotante.

I. INTRODUCCIÓN

La normalización mediante la puntuación z, definida como $y_i = (x_i - \mu)/\sigma$, es una operación recurrente en el preprocesamiento de señales y datos. Su costo por elemento es bajo, pero crece linealmente con el tamaño del conjunto y exige al menos dos recorridos completos del arreglo: uno para obtener los estadísticos descriptivos y otro para aplicar la transformación.

Las arquitecturas x86-64 modernas ofrecen extensiones SIMD (*Single Instruction, Multiple Data*) que permiten operar sobre varios datos con una sola instrucción. SSE trabaja sobre registros XMM de 128 bits, capaces de alojar cuatro valores de punto flotante de simple precisión, mientras que AVX2 amplía los registros a 256 bits (YMM) y permite ocho por operación, lo que establece una aceleración teórica de hasta ocho veces. Ese límite rara vez se alcanza: las reducciones horizontales, el procesamiento del remanente y la saturación del ancho de banda de memoria introducen costos que la vectorización no elimina.

Este trabajo implementa el kernel directamente en lenguaje ensamblador, sin recurrir a la vectorización automática del compilador, con el objetivo de cuantificar la brecha entre la aceleración teórica y la medida e identificar sus causas. Se desarrollaron dos versiones funcionalmente equivalentes de las rutinas `sum_array`, `compute_stats` y `normalize_array`.

La sección II describe el entorno de pruebas y la sección III la arquitectura del programa. Las secciones IV a VII presentan ambas implementaciones y su verificación con GDB. La sección VIII reporta los casos de prueba, la sección IX los resultados de rendimiento y precisión, y la sección X analiza sus causas.

II. ENTORNO DE PRUEBAS

El desarrollo se realizó en dos equipos, descritos en la tabla I. Ambos soportan AVX2, lo que se confirmó con `lscpu` antes de iniciar, tal como exige el enunciado. El equipo 1 se utilizó para desarrollar y depurar la versión escalar; todas las mediciones de rendimiento y de precisión de las secciones VIII y IX se realizaron en el equipo 2, de modo que ambas versiones se comparan sobre el mismo hardware.

```
1 $ lscpu | grep -E "Nombre_del_modelo"
2 Nombre del modelo: Intel(R) Core(TM) i3-10110U
   CPU @ 2.10GHz
3 $ lscpu | grep -o 'avx[0-9a-z]*' | sort -u
4 avx
5 avx2
6
7 $ lscpu | grep -E "Nombre_del_modelo"
8 Nombre del modelo: AMD Ryzen 7 7730U with Radeon
   Graphics
9 $ lscpu | grep -o 'avx[0-9a-z]*' | sort -u
10 avx
11 avx2
```

Listing 1. Verificación del soporte de AVX2 en los equipos de prueba.

III. ARQUITECTURA DEL PROGRAMA

El programa sigue la arquitectura híbrida recomendada: un *driver* en C se encarga de leer el archivo de entrada, reservar memoria alineada a 32 bytes con `aligned_alloc`, medir el tiempo y escribir los resultados, mientras que los tres kernels de cómputo están escritos enteramente en NASM. Ambas

Cuadro I
EQUIPOS Y HERRAMIENTAS UTILIZADOS.

	Equipo 1	Equipo 2
CPU	Intel Core i3-10110U	AMD Ryzen 7 7730U
Caché L1d	32 KiB/núcleo	32 KiB/núcleo
Caché L2	256 KiB/núcleo	512 KiB/núcleo
Caché L3	4 MiB	16 MiB
Uso	Desarrollo escalar	Mediciones

versiones exportan las mismas firmas, declaradas en stats.h, de modo que el mismo *driver* se enlaza con una u otra para producir `norm_scalar` y `norm_vector`.

Los diagramas que documentan la estructura de ambos programas se entregan como archivos adjuntos a este informe. Incluyen el diagrama de bloques de la arquitectura de software, con los módulos del programa y su comunicación según la ABI System V; el diagrama de flujo de control del bucle principal de cada kernel, que muestra la inicialización del acumulador, el bucle vectorial de ocho en ocho, la transición al bucle de remanente y la condición de salida; la tabla de asignación de registros en cada etapa del algoritmo; y el diagrama de memoria, con la alineación de la dirección base a 32 bytes y el avance del puntero en cada iteración (4 bytes en la versión escalar y 32 bytes en la vectorial).

III-A. Convención de llamada

La comunicación entre C y NASM respeta la ABI System V AMD64. Los argumentos enteros y punteros llegan en `rdi`, `rsi`, `rdx`, `rcx`, `r8` y `r9`, en ese orden; los de punto flotante en `xmm0` y `xmm1`, y el valor de retorno de `sum_array` se devuelve en `xmm0` (tabla II).

Cuadro II
PASO DE ARGUMENTOS SEGÚN LA ABI SYSTEM V.

Función	Registro	Argumento
<code>sum_array</code>	<code>rdi</code> , <code>esi</code>	<code>arr</code> , <code>n</code>
<code>compute_stats</code>	<code>rdi</code> , <code>esi</code> <code>rdx</code> , <code>rcx</code> <code>r8</code> , <code>r9</code>	<code>arr</code> , <code>n</code> <code>*mean</code> , <code>*var</code> <code>*min</code> , <code>*max</code>
<code>normalize_array</code>	<code>rdi</code> , <code>rsi</code> , <code>edx</code> <code>xmm0</code> , <code>xmm1</code>	<code>in</code> , <code>out</code> , <code>n</code> <code>mean</code> , <code>stddev</code>

Los registros `rbx`, `rbp` y `r12`–`r15` son *callee-saved*: si la función los modifica, debe restaurarlos antes de retornar. Los kernels escalares solo usan registros volátiles y no necesitan preservar nada. El `compute_stats` vectorial, en cambio, llama internamente a `sum_array`, por lo que guarda sus argumentos en registros *callee-saved* que sobreviven a esa llamada. Por ello los apila al entrar y los desapila en orden inverso al salir, y resta 8 bytes adicionales a `rsp` para que la pila quede alineada a 16 bytes en el momento del `call`, como exige la ABI:

```

1  push rbx
2  push rbp
3  push r12
4  push r13
5  push r14
6  push r15

```

```

7  sub  rsp, 8      ; 6 pushes dejan rsp en 8 mod
   16
8  ...
9  call sum_array

```

Listing 2. Preservación de registros en `compute_stats` vectorial.

Finalmente, todas las funciones vectoriales ejecutan `vzeroupper` antes de `ret`. Esta instrucción pone en cero la mitad alta de los registros YMM y evita la penalización por transición entre código AVX y código SSE heredado que pueda ejecutar el *driver* o la biblioteca de C a continuación.

IV. IMPLEMENTACIÓN ESCALAR

La versión escalar utiliza únicamente las instrucciones escalares de SSE (`movss`, `addss`, `subss`, `mulss`, `divss`, `comiss`), que operan sobre un solo float en el carril bajo de un registro XMM. Se procesa un elemento por iteración y el puntero avanza 4 bytes en cada paso.

IV-A. Asignación de registros

La tabla III resume el uso de registros en el kernel escalar.

Cuadro III
USO DE REGISTROS EN EL KERNEL ESCALAR.

Registro	Función	Contenido
<code>xmm0</code>	<code>sum_array</code>	Acumulador (valor de retorno)
<code>xmm1</code>	<code>sum_array</code>	Temporal: <code>arr[i]</code>
<code>xmm0</code>	<code>compute_stats</code>	Suma, luego media
<code>xmm1</code>	<code>compute_stats</code>	Mínimo acumulado
<code>xmm2</code>	<code>compute_stats</code>	Máximo acumulado
<code>xmm3</code>	<code>compute_stats</code>	Temporal: <code>arr[i]</code>
<code>xmm4</code>	<code>compute_stats</code>	<code>n</code> convertido a float
<code>xmm5</code>	<code>compute_stats</code>	Acumulador de $\sum (x - \mu)^2$
<code>xmm0</code>	<code>normalize_array</code>	<code>mean</code>
<code>xmm1</code>	<code>normalize_array</code>	<code>stddev</code>
<code>xmm2</code>	<code>normalize_array</code>	<code>in[i]</code> , luego <code>y_i</code>
<code>xmm3</code>	<code>normalize_array</code>	Constante 0,0 (comparación)
<code>eax</code>	todas	Índice del elemento actual

IV-B. Estructura común de los bucles escalares

Las tres funciones utilizan el mismo patrón de flujo: inicialización, condición de parada al inicio del bucle, cuerpo, incremento del índice y salto incondicional de regreso. Al evaluar la condición antes del cuerpo, el caso $N = 0$ queda cubierto sin código adicional: el bucle termina antes de acceder a memoria.

IV-B1. `sum_array`: Esta función recorre el arreglo de entrada acumulando la suma en `xmm0`, que es además el registro de retorno.

```

1  sum_array:
2  xor  eax, eax      ; i = 0
3  pxor  xmm0, xmm0   ; acumulador = 0.0
4  .loop:
5  cmp  eax, esi      ; compara i con n
6  jge  .done         ; si i >= n, salir
7  movss xmm1, [rdi + rax*4] ; xmm1 = arr[i]
8  addss xmm0, xmm1   ; acumulador += arr[i]
9  inc  eax           ; i = i + 1
10 jmp  .loop         ; repetir

```

```

11 .done:
12     ret                ; resultado en xmm0

```

Listing 3. Función sum_array escalar.

IV-B2. compute_stats: Esta es la función más compleja del kernel escalar: calcula media, varianza, mínimo y máximo mediante dos recorridos completos del arreglo. Primero atiende el caso $N = 0$, escribiendo cero en las cuatro salidas sin dividir entre N , e inicializa los acumuladores. El mínimo y el máximo se inicializan con `arr[0]` y no con cero: en un arreglo de valores exclusivamente negativos, un máximo inicial de cero nunca se actualizaría.

```

1  compute_stats:
2      test    esi, esi                ; n == 0?
3      jne     .valid_n
4      pxor    xmm0, xmm0
5      movss   [rdx], xmm0            ; mean = 0
6      movss   [rcx], xmm0            ; var = 0
7      movss   [r8], xmm0             ; min = 0
8      movss   [r9], xmm0            ; max = 0
9      ret
10 .valid_n:
11     xor     eax, eax
12     pxor    xmm0, xmm0            ; suma = 0.0
13     movss   xmm1, [rdi]            ; min = arr[0]
14     movss   xmm2, [rdi]            ; max = arr[0]

```

Listing 4. Caso $N = 0$ e inicialización.

El primer bucle acumula la suma y actualiza el mínimo y el máximo mediante comparaciones con `comiss` y saltos condicionales. Al terminar el recorrido se divide la suma entre N para obtener la media.

```

1  .loop1:
2      cmp     eax, esi
3      jge     .after_loop1
4      movss   xmm3, [rdi + rax*4]    ; arr[i]
5      addss   xmm0, xmm3            ; suma += arr[i]
6      comiss  xmm3, xmm1
7      jae     .skip_min
8      movss   xmm1, xmm3            ; min = arr[i]
9  .skip_min:
10     comiss  xmm3, xmm2
11     jbe     .skip_max
12     movss   xmm2, xmm3            ; max = arr[i]
13 .skip_max:
14     inc     eax
15     jmp     .loop1
16 .after_loop1:
17     movss   [r8], xmm1            ; *min
18     movss   [r9], xmm2            ; *max
19     cvtsi2ss xmm4, esi            ; (float) n
20     divss   xmm0, xmm4            ; mean = suma / n
21     movss   [rdx], xmm0            ; *mean

```

Listing 5. Primer recorrido: suma, mínimo y máximo.

La varianza poblacional requiere conocer la media, por lo que se calcula en un segundo recorrido que acumula $(x_i - \mu)^2$:

```

1  xor     eax, eax
2  pxor    xmm5, xmm5            ; suma_sq_diff = 0.0
3  .loop2:
4      cmp     eax, esi
5      jge     .after_loop2
6      movss   xmm3, [rdi + rax*4]    ; arr[i]
7      subss   xmm3, xmm0            ; arr[i] - mean

```

```

8      mulss   xmm3, xmm3            ; al cuadrado
9      addss   xmm5, xmm3            ; acumula
10     inc     eax
11     jmp     .loop2
12 .after_loop2:
13     divss   xmm5, xmm4            ; var = suma_sq_diff / n
14     movss   [rcx], xmm5            ; *var
15     ret

```

Listing 6. Segundo recorrido: varianza.

IV-B3. normalize_array: Esta función transforma cada elemento en su puntuación z , $y_i = (x_i - \mu)/\sigma$. Primero resuelve el caso borde $\sigma = 0$, que se presenta cuando todos los valores son iguales o cuando $N = 1$: dividir entre cero produciría infinitos o NaN. Siguiendo el contrato definido en `stats.h`, en ese caso la entrada se copia sin modificar en la salida.

```

1  normalize_array:
2      xor     eax, eax            ; i = 0
3      xorps   xmm3, xmm3          ; xmm3 = 0.0
4      comiss  xmm1, xmm3          ; stddev == 0?
5      je     .loop_zero_std      ; si, camino especial
6  .loop:
7      cmp     eax, edx
8      jge     .done
9      movss   xmm2, [rdi + rax*4]  ; in[i]
10     subss   xmm2, xmm0           ; in[i] - mean
11     divss   xmm2, xmm1           ; / stddev
12     movss   [rsi + rax*4], xmm2  ; out[i]
13     inc     eax
14     jmp     .loop
15 .loop_zero_std:
16     cmp     eax, edx
17     jge     .done
18     movss   xmm2, [rdi + rax*4]  ; in[i]
19     movss   [rsi + rax*4], xmm2  ; out[i] = in[i]
20     inc     eax
21     jmp     .loop_zero_std
22 .done:
23     ret

```

Listing 7. `normalize_array` escalar.

Una primera versión escribía 0,0 en la salida en lugar de copiar la entrada. La batería de pruebas automática detectó la discrepancia en los casos $N = 1$ y “todos iguales”, y se corrigió en ambas versiones.

V. SESIÓN DE GDB ESCALAR

Se verificó el comportamiento del kernel escalar para el caso $N = 16$, generado con semilla fija:

```

1  python3 tools/generate_test_data.py --outdir data --seed 42
2  gdb --args ./bin/norm_scalar data/input_n16.dat data/output_scalar.dat 1

```

Se fijó un punto de ruptura a la entrada de `normalize_array` para leer los parámetros que llegan en `xmm0` y `xmm1`:

```

1  (gdb) break normalize_array
2  Breakpoint 1 at 0x18a8: file asm/scalar/stats_scalar.asm, line 27.
3  (gdb) run
4  Breakpoint 1, normalize_array () at asm/scalar/stats_scalar.asm:27

```

```

5 27      xor     eax, eax                ; i = 0
6 (gdb) print $xmm0.v4_float[0]
7 $1 = 1.65170157
8 (gdb) print $xmm1.v4_float[0]
9 $2 = 50.2191734

```

Los valores corresponden a μ y σ . Luego se fijaron puntos de ruptura en la carga de `in[i]` y en la escritura de `out[i]`, para observar el resultado de la primera iteración antes de guardarse:

```

1 (gdb) break stats_scalar.asm:36
2 (gdb) break stats_scalar.asm:39
3 (gdb) continue
4 Breakpoint 2, normalize_array[loop] () at asm/
  scalar/stats_scalar.asm:36
5 36      movss   xmm2, [rdi + rax*4]    ; in[i]
6 (gdb) continue
7 Breakpoint 3, normalize_array[loop] () at asm/
  scalar/stats_scalar.asm:39
8 39      movss   [rsi + rax*4], xmm2    ; out[i
9 ] = resultado
10 (gdb) print $xmm2.v4_float[0]
$3 = 1.8289876

```

Por último se eliminaron esos puntos de ruptura, se dejó terminar la función y se inspeccionó el arreglo de salida completo:

```

1 (gdb) delete 2 3
2 (gdb) finish
3 main (...) at src/driver.c:140
4 140      clock_gettime(CLOCK_MONOTONIC, &
5 t1);
6 (gdb) print out[0]@16
7 $4 = {1.8289876, -0.726547778, -0.548789561,
8 -0.154135138,
9 -1.26958346, -1.50674331, -0.129646033,
10 -1.12048507,
11 0.643401623, -0.283185035, 1.29201519,
12 0.764674425,
13 -0.780147731, 1.29034889, 1.18084717,
14 -0.481011987}

```

El primer elemento de entrada es $x_0 = 93,501945$. Con los valores de μ y σ leídos en el registro:

$$y_0 = \frac{93,501945 - 1,65170157}{50,2191734} = 1,8289876$$

que coincide con \$3 y con el primer elemento de \$4. La referencia en doble precisión da $\mu = 1,651702$ y $\sigma = 50,219174$, iguales a los del kernel hasta la séptima cifra significativa.

VI. IMPLEMENTACIÓN VECTORIAL

VI-A. Elección de AVX2 frente a SSE y AVX-512

Se eligió AVX2 porque ofrece el mayor grado de paralelismo de datos disponible en ambos equipos. Sus registros YMM de 256 bits alojan ocho valores de simple precisión, frente a los cuatro de los registros XMM de SSE, lo que duplica el trabajo por instrucción. AVX-512 lo duplicaría de nuevo con registros ZMM de 512 bits, pero ninguno de los procesadores utilizados lo soporta, como muestra la salida de `lscpu` de la sección II. Además, AVX2 cuenta con instrucciones empaquetadas de mínimo y máximo (`vminps`, `vmaxps`) y de difusión (`vbroadcastss`), que el kernel aprovecha directamente.

VI-B. Asignación de registros

La tabla IV muestra los registros utilizados en el kernel vectorial.

Cuadro IV
USO DE REGISTROS EN EL KERNEL VECTORIAL.

Registro	Función	Contenido
ymm0	sum_array	Acumulador (8 sumas parciales)
ymm1	todas	Bloque de 8 floats de entrada
ymm2	compute_stats	Desviaciones al cuadrado
ymm4	compute_stats	Acumulador de $\sum (x - \mu)^2$
ymm5	compute_stats	Mínimos parciales
ymm7	compute_stats	Media en los 8 carriles
ymm8	compute_stats	Máximos parciales
rbx	compute_stats	Puntero al arreglo (preservado)
r12d	compute_stats	N (preservado)
ymm2	normalize_array	Media en los 8 carriles
ymm3	normalize_array	Desviación en los 8 carriles
eax	todas	Índice del elemento actual
ecx	todas	Límite del bucle vectorial

VI-C. Estructura común de los bucles vectoriales

Las tres funciones comparten la misma estructura: un bucle que procesa ocho elementos por iteración hasta agotar el mayor múltiplo de ocho contenido en N , seguido de un bucle escalar que procesa los elementos restantes. El límite del bucle vectorial se obtiene apagando los tres bits menos significativos de N , lo que equivale a $\lfloor N/8 \rfloor \cdot 8$ sin realizar una división:

```

1      mov ecx, esi          ; ecx = N
2      and ecx, ~7          ; ecx = N redondeado a
                           multiplo de 8

```

Listing 8. Cálculo del límite del bucle vectorial.

VI-D. sum_array

Cada iteración ejecuta una sola instrucción que actualiza los ocho carriles del acumulador. El índice avanza de ocho en ocho, lo que equivale a 32 bytes por iteración frente a los 4 bytes de la versión escalar. Al terminar el bucle, `ymm0` contiene ocho sumas parciales independientes, que se combinan como se describe en la sección VI-F.

```

1      vxorps ymm0, ymm0, ymm0    ; acumulador en
2      .sum_vec_loop:             cero
3      cmp eax, ecx
4      jge .sum_reduce
5      vmovaps ymm1, [rdi + rax*4] ; 8 floats
6      vaddps ymm0, ymm0, ymm1     ; 8 sumas
7      add eax, 8                  ; avanza 32 bytes
8      jmp .sum_vec_loop

```

Listing 9. Bucle vectorial de acumulación.

VI-E. *compute_stats*

La función realiza dos pasadas sobre el arreglo. La primera obtiene la media reutilizando `sum_array`; la segunda calcula en un solo recorrido la suma de cuadrados de las desviaciones, el mínimo y el máximo. A diferencia de la versión escalar, que obtiene mínimo y máximo en la primera pasada, aquí se agrupan con la varianza porque `vmnps` y `vmaxps` operan sobre el mismo bloque ya cargado en `ymm1`, sin necesidad de saltos condicionales.

```
1  vbroadcastss ymm7, xmm6      ; media en los 8
   carriles
2  vxorps ymm4, ymm4, ymm4      ; acumulador de
   cuadrados en cero
3  vbroadcastss ymm5, [rbx]      ; min
   inicializado con arr[0]
4  vmovaps ymm8, ymm5           ; max
   inicializado con arr[0]
```

Listing 10. Inicialización de la segunda pasada.

La media se difunde a los ocho carriles de `ymm7` mediante `vbroadcastss`, porque `vsubps` opera carril a carril y necesita la media en cada uno de ellos. Los acumuladores de mínimo y máximo se inicializan con `arr[0]` por la misma razón expuesta en la versión escalar.

```
1  .stats_loop:
2  cmp eax, ecx
3  jge .stats_reduce
4  vmovaps ymm1, [rbx + rax*4]
5  vsubps ymm2, ymm1, ymm7      ; x - mu
6  vmulps ymm2, ymm2, ymm2      ; (x - mu)^2
7  vaddps ymm4, ymm4, ymm2      ; acumula
8  vmnps ymm5, ymm5, ymm1      ; min por carril
9  vmaxps ymm8, ymm8, ymm1      ; max por carril
10 add eax, 8
11 jmp .stats_loop
```

Listing 11. Cuerpo del bucle: varianza, mínimo y máximo en un solotamaños de entrada.

VI-F. *Reducción horizontal de la suma*

Las instrucciones empaquetadas operan carril a carril (verticalmente), por lo que combinar los ocho carriles de un mismo registro requiere una reducción horizontal:

```
1  .sum_reduce:
2  vextractf128 xmm2, ymm0, 1    ; 8 -> 4: aisla
   la mitad alta
3  vaddps xmm0, xmm0, xmm2      ; y la
   suma con la baja
4  vhaddps xmm0, xmm0, xmm0      ; 4 -> 2: suma
   pares adyacentes
5  vhaddps xmm0, xmm0, xmm0      ; 2 -> 1
```

Listing 12. Reducción $8 \rightarrow 1$ de la suma.

`vextractf128` copia los 128 bits altos de `ymm0` a un registro XMM, de modo que un `vaddps` de 128 bits suma ambas mitades. Se eligió este primer paso porque `vhaddps` sobre YMM opera dentro de cada mitad de 128 bits por separado y nunca cruza entre ellas, de modo que no puede completar la reducción por sí solo. Los dos `vhaddps` siguientes suman pares de carriles adyacentes hasta dejar el total en el carril 0.

La alternativa era sustituir los `vhaddps` por `vmovhlps/vshufps` seguidos de `vaddps`, técnica que en

varias microarquitecturas tiene menor latencia que `vhaddps`, internamente descompuesta en varias operaciones. Se prefirió `vhaddps` por legibilidad: la reducción se ejecuta una sola vez por llamada, por lo que su costo es despreciable frente a las $N/8$ iteraciones del bucle (sección X-B).

VI-G. *Reducción del mínimo y del máximo*

AVX2 no ofrece un equivalente horizontal de `vhaddps` para mínimo y máximo, por lo que la combinación entre carriles debe hacerse reordenándolos explícitamente y aplicando después la operación vertical:

```
1  vextractf128 xmm2, ymm5, 1    ; 8 -> 4
2  vmnps xmm5, xmm5, xmm2
3  vmovhlps xmm2, xmm2, xmm5      ; 4 -> 2
4  vmnps xmm5, xmm5, xmm2
5  vshufps xmm2, xmm5, xmm5, 0x55 ; 2 -> 1
6  vminss xmm5, xmm5, xmm2
```

Listing 13. Reducción del mínimo.

`vmovhlps` traslada la mitad alta de `xmm5` a la parte baja de `xmm2`, de modo que el `vmnps` siguiente compare el carril 0 con el 2 y el carril 1 con el 3. El inmediato `0x55` de `vshufps` replica el carril 1 en las cuatro posiciones, dejando el último par listo para compararse con `vminss`. La reducción del máximo es idéntica sustituyendo `vmnps/vminss` por `vmaxps/vmaxss`.

VI-H. *Manejo del remanente*

Cuando N no es múltiplo de ocho, el bucle vectorial deja sin procesar entre uno y siete elementos. Omitirlos produciría resultados silenciosamente incorrectos para siete de cada ocho

```
1  .sum_scalar_tail:
2  cmp eax, esi                  ; esi = N total,
   no el redondeado
3  jge .sum_done
4  vmovss xmm1, [rdi + rax*4]    ; un solo float
5  vaddss xmm0, xmm0, xmm1      ; suma escalar
6  inc eax
7  jmp .sum_scalar_tail
```

Listing 14. Bucle de remanente de `sum_array`.

La diferencia con el bucle vectorial está en el límite: el bucle principal se detiene en `ecx`, el múltiplo de ocho, mientras que el de remanente continúa hasta `esi`, el N verdadero. Las instrucciones pasan de la forma empaquetada a la escalar (`vmovss` y `vaddss` en lugar de `vmovaps` y `vaddps`) y el índice avanza de uno en uno. Se usan las formas VEX de las instrucciones escalares para no mezclar codificaciones SSE y AVX dentro de la función.

El remanente se acumula sobre el resultado ya reducido: el bucle escalar se ejecuta después de la reducción horizontal y suma sus elementos al escalar que esta produjo, no a los acumuladores parciales. Para $N < 8$, como en el caso $N = 7$, el bucle vectorial no ejecuta ninguna iteración y todo el trabajo recae en el remanente.

VI-I. *normalize_array*

Esta función aplica $y_i = (x_i - \mu)/\sigma$ a cada elemento. A diferencia de las dos anteriores, no acumula ningún valor: cada resultado depende exclusivamente de su propia entrada, por lo que los ocho carriles operan de forma independiente y no se requiere reducción horizontal. La media y la desviación llegan como escalares en `xmm0` y `xmm1`; antes de entrar al bucle se difunden a los ocho carriles:

```
1 .broadcast:
2   vbroadcastss ymm2, xmm4 ; media en los 8
   carriles
3   vbroadcastss ymm3, xmm5 ; desviacion en los
   8 carriles
```

Listing 15. Difusión de los parámetros escalares.

```
1 .loop:
2   cmp eax, ecx
3   jge .tail
4   vmovaps ymm1, [rdi + rax*4] ; 8 floats de
   entrada
5   vsubps ymm1, ymm1, ymm2 ; x - mu
6   vdivps ymm1, ymm1, ymm3 ; / sigma
7   vmovaps [rsi + rax*4], ymm1 ; 8 floats de
   salida
8   add eax, 8
9   jmp .loop
```

Listing 16. Bucle vectorial de normalización.

VI-J. *Alineación de memoria*

El camino principal de las tres funciones utiliza `vmovaps`, que exige que la dirección accedida sea múltiplo de 32 bytes y genera una excepción de protección general en caso contrario. `vmovups` no tiene esa restricción, pero se optó por la forma alineada porque el *driver* garantiza la alineación: reserva ambos arreglos con `aligned_alloc(32, ...)`, redondeando el tamaño a un múltiplo de 32. Como el índice avanza de 8 en 8, cada acceso del bucle vectorial cae en una dirección base + $32k$ y conserva la alineación. El remanente usa `vmovss`, que no impone restricción alguna.

VI-K. *Casos borde*

VI-K1. $N = 0$: `compute_stats` comprueba el contador antes de cualquier acceso a memoria y, si es nulo, escribe cero en las cuatro salidas sin dividir entre N . `normalize_array` retorna de inmediato.

VI-K2. *Desviación estándar nula*: Si $\sigma = 0$, la normalización dividiría entre cero. La función detecta el caso antes de entrar al bucle y copia la entrada en la salida sin modificarla:

```
1   vxorps xmm2, xmm2, xmm2
2   vucomiss xmm5, xmm2
3   jp .broadcast ; NaN: no es igual a cero
4   je .copy ; sigma == 0: copiar sin
   normalizar
```

Listing 17. Detección de $\sigma = 0$ y de NaN.

Cuando `vucomiss` compara un NaN, el resultado es no ordenado y la instrucción activa simultáneamente los indicadores de paridad, cero y acarreo. Un `je` aislado interpretaría entonces un NaN como igualdad con cero y desviaría la ejecución al

camino de copia. El `jp` previo detecta el caso no ordenado y evita ese falso positivo.

```
1 .copy:
2   xor eax, eax
3 .copy_next:
4   cmp eax, edx
5   jge .done
6   vmovss xmm1, [rdi + rax*4] ; in[i]
7   vmovss [rsi + rax*4], xmm1 ; out[i] = in[i]
8   inc eax
9   jmp .copy_next
```

Listing 18. Copia de la entrada cuando $\sigma = 0$.

VII. SESIÓN DE GDB VECTORIAL

Se verificó manualmente el kernel vectorial con el caso $N = 16$ exigido por el enunciado, usando la misma entrada que la sesión escalar (sección V), de modo que ambas sesiones sean comparables:

```
1 gdb --args ./bin/norm_vector data/input_n16.dat
   data/output_vector.dat 1
```

La instrucción de acumulación del bucle vectorial de `sum_array` es `vaddps ymm0, ymm0, ymm1`, en la línea 22 del archivo, donde `ymm0` es el acumulador y `ymm1` contiene los ocho floats recién cargados. Se fijaron dos puntos de ruptura, en la línea de la instrucción y en la siguiente, de modo que cada iteración quede registrada antes y después de ejecutarla:

```
1 (gdb) break statsvar_vector.asm:22
2 Breakpoint 1 at 0x1ad8: file asm/vector/
   statsvar_vector.asm, line 22.
3 (gdb) break statsvar_vector.asm:23
4 Breakpoint 2 at 0x1adc: file asm/vector/
   statsvar_vector.asm, line 23.
5 (gdb) run
```

VII-A. *Registros YMM en el bucle de acumulación*

```
1 Breakpoint 1, sum_array.sum_vec_loop () at
   statsvar_vector.asm:22
2 22 vaddps ymm0, ymm0, ymm1
3 (gdb) print $ymm0.v8_float
4 $1 = {0, 0, 0, 0, 0, 0, 0, 0}
5 (gdb) print $ymm1.v8_float
6 $2 = {93.5019455, -34.8349266, -25.9080582,
   -6.08883762,
   -62.1057281, -74.0157013, -4.85901499,
   -54.6181297}
7 (gdb) continue
8 Breakpoint 2, sum_array.sum_vec_loop () at
   statsvar_vector.asm:23
9 23 add eax, 8
10 (gdb) print $ymm0.v8_float
11 $3 = {93.5019455, -34.8349266, -25.9080582,
   -6.08883762,
   -62.1057281, -74.0157013, -4.85901499,
   -54.6181297}
```

Listing 19. Iteración 1. `$1` y `$2` son el acumulador y el bloque recién cargado antes de `vaddps`; `$3` es el acumulador inmediatamente después.

```
1 Breakpoint 1, sum_array.sum_vec_loop () at
   statsvar_vector.asm:22
2 (gdb) print $ymm0.v8_float
3 $4 = {93.5019455, -34.8349266, -25.9080582,
   -6.08883762,
```

```

4      -62.1057281, -74.0157013, -4.85901499,
      -54.6181297}
5 (gdb) print $ymm1.v8_float
6 $5 = {33.9627991, -12.5696163, 66.5356369,
      40.0530205,
7      -37.5266724, 66.4519577, 60.9528732,
      -22.504324}
8 (gdb) continue
9 Breakpoint 2, sum_array.sum_vec_loop () at
      statsvar_vector.asm:23
10 (gdb) print $ymm0.v8_float
11 $6 = {127.464745, -47.404541, 40.6275787,
      33.9641838,
12      -99.6324005, -7.56374359, 56.0938568,
      -77.1224518}

```

Listing 20. Iteración 2. \$4 es el acumulador antes de la segunda vaddps, \$5 el segundo bloque de datos y \$6 el acumulador después.

En la primera iteración, una sola instrucción escribe en los ocho carriles: el acumulador pasa de contener ceros a contener los ocho primeros elementos del arreglo. Estos coinciden con los primeros valores de entrada listados en la sesión escalar, lo que confirma que vmovaps cargó el bloque correcto. En la segunda iteración se observa el paralelismo de datos: cada carril acumula de forma independiente el elemento que le corresponde, sin interacción con los carriles vecinos.

carril 0 : $93,501946 + 33,962799 = 127,464745$
carril 1 : $-34,834927 + (-12,569616) = -47,404541$
carril 4 : $-62,105728 + (-37,526672) = -99,632401$

Los valores calculados a mano coinciden con los observados en \$6. Con $N = 16$ el bucle vectorial completa el recorrido en dos iteraciones, frente a las dieciséis que requiere la versión escalar. Al terminar, ymm0 contiene ocho sumas parciales que la reducción horizontal de la sección VI-F combina en un único valor.

VII-B. Memoria del arreglo de salida

Para inspeccionar el resultado de la normalización se eliminaron los puntos de ruptura del bucle y se fijó uno en el *driver*, en la línea posterior a la llamada a `normalize_array`, lo que permite referirse al arreglo y a los estadísticos por su nombre en C.

```

1 (gdb) delete 1 2
2 (gdb) break driver.c:147
3 (gdb) continue
4 Breakpoint 3, main (...) at src/driver.c:147
5 147      clock_gettime(CLOCK_MONOTONIC, &t1);
6 (gdb) print mean
7 $7 = 1.65170169
8 (gdb) print var
9 $8 = 2521.96533
10 (gdb) x/16fw &out[0]
11 0x555555559520: 1.8289876   -0.726547778
      -0.548789561 -0.154135138
12 0x555555559530: -1.26958346  -1.50674331
      -0.129646033 -1.12048507
13 0x555555559540: 0.643401623 -0.283185035
      1.29201519   0.764674425
14 0x555555559550: -0.780147731  1.29034889
      1.18084717   -0.481011987
15 (gdb) print &out[0]
16 $9 = (float *) 0x555555559520

```

Listing 21. Estadísticos y contenido del arreglo de salida tras el bucle de normalización.

Con $\mu = 1,65170169$ y $\sigma = \sqrt{2521,96533} = 50,219173$, el primer elemento verifica el cálculo:

$$y_0 = \frac{93,501946 - 1,651702}{50,219173} = 1,8289876$$

resultado que coincide con el valor leído en memoria.

Los dieciséis valores de salida son idénticos a los obtenidos por la versión escalar en la sección V. La media, en cambio, difiere en las últimas cifras: la escalar obtuvo 1,65170157 y la vectorial 1,65170169. La diferencia proviene del orden de las sumas: la versión escalar acumula los dieciséis elementos en secuencia, mientras que la vectorial suma primero en ocho carriles y luego los combina en la reducción horizontal. Es una muestra, a pequeña escala, de las diferencias de redondeo que anticipa el enunciado y que la sección X-E analiza para tamaños grandes; a este tamaño, la diferencia no alcanza a modificar ningún valor del arreglo normalizado en precisión simple.

Por último, la dirección base del arreglo, 0x555555559520, termina en 0x20: como $0x9520 = 38176 = 32 \times 1193$, es múltiplo de 32 bytes, condición que habilita el uso de vmovaps en el camino principal del kernel.

VIII. CASOS DE PRUEBA

La correctitud se verificó automáticamente con `tools/verify.py`, que recalcula en doble precisión los estadísticos y el arreglo normalizado de cada entrada y los compara con la salida de cada kernel. Los estadísticos se aceptan si su error relativo es menor que 10^{-4} . El arreglo se acepta si $|y_i - y_i^{\text{ref}}| \leq 10^{-4} + 10^{-4}|y_i^{\text{ref}}|$: se añade un término absoluto porque los z-scores cercanos a cero harían crecer el error relativo puro sin que exista un error numérico real. La batería completa se ejecuta con `make test`; la tabla V resume los resultados.

El caso `input_extreme` falla en ambas versiones por una limitación del formato `float32`, no por un error de los kernels. Su varianza de referencia es $\sigma^2 \approx 2,9 \times 10^{76}$, que supera ampliamente el máximo representable en simple precisión ($\approx 3,4 \times 10^{38}$). Al elevar al cuadrado la desviación de $3,4 \times 10^{38}$, el producto se desborda a $+\infty$, y con él la varianza y la desviación. Los estadísticos que no elevan al cuadrado sí son correctos: la suma, la media, el mínimo y el máximo pasan la verificación. En el arreglo, cada $(x_i - \mu)/\infty$ da ± 0 , de modo que los dos elementos de $\pm 3,4 \times 10^{38}$, cuya referencia es ± 2 , quedan fuera de tolerancia, mientras que los otros seis, cuya referencia es prácticamente cero, la cumplen. Como el enunciado exige realizar todo el cómputo en `float32`, el resultado es el esperado para este formato. Un valor extremo más moderado, como 10^{18} , cuyo cuadrado (10^{36}) sí es representable, se procesa correctamente.

Cuadro V
CASOS DE PRUEBA: REFERENCIA EN DOBLE PRECISIÓN Y RESULTADO DE CADA VERSIÓN.

Caso	Qué verifica	N	μ_{ref}	σ_{ref}	Escalar	Vectorial
input_n0	Arreglo vacío, sin división entre cero	0	0	0	PASA	PASA
input_n1	$N = 1$, $\sigma = 0$ (copia de la entrada)	1	54,791210	0	PASA	PASA
input_n7	$N < 8$: solo remanente	7	31,767089	55,323306	PASA	PASA
input_n8	Un bloque vectorial exacto	8	0,333555	52,059254	PASA	PASA
input_n15	1 bloque + 7 de remanente	15	8,263707	59,979412	PASA	PASA
input_n16	Dos bloques exactos	16	1,651702	50,219174	PASA	PASA
input_n1000	Caso general	1000	-0,943230	58,142978	PASA	PASA
input_all_equal	Todos iguales, $\sigma = 0$	64	7,5	0	PASA	PASA
input_negative	Solo valores negativos	100	-229,168332	142,986288	PASA	PASA
input_extreme	$\pm 3,4 \times 10^{38}$, subnormales, ± 0	8	0	$1,7 \times 10^{38}$	FALLA	FALLA

IX. RESULTADOS

IX-A. Metodología de medición

Se midió exclusivamente el tiempo de los tres kernels, excluyendo la lectura y escritura de archivos. El *driver* rodea la secuencia `sum_array`, `compute_stats` y `normalize_array` con `clock_gettime(CLOCK_MONOTONIC)` y con la lectura del contador de ciclos (`__rdtsc`), y repite la medición 30 veces por tamaño para reportar el promedio y la desviación estándar. El proceso se automatizó con `tools/run_benchmark.py` para los cuatro tamaños grandes del enunciado, en el equipo 2.

IX-B. Tiempos y speedup

La tabla VI reporta los tiempos medidos y el speedup, definido como $S = t_{\text{escalar}}/t_{\text{vectorial}}$. La fig. 1 lo grafica en función de N .

Los ciclos se obtienen con `__rdtsc`, que cuenta a la frecuencia nominal constante del contador TSC (unos 2,0 GHz en este equipo, según la relación entre ciclos y milisegundos de la tabla), no a la frecuencia instantánea del núcleo; por eso son proporcionales al tiempo de pared. La columna de ciclos por elemento divide los ciclos entre N e incluye los cuatro recorridos del arreglo.

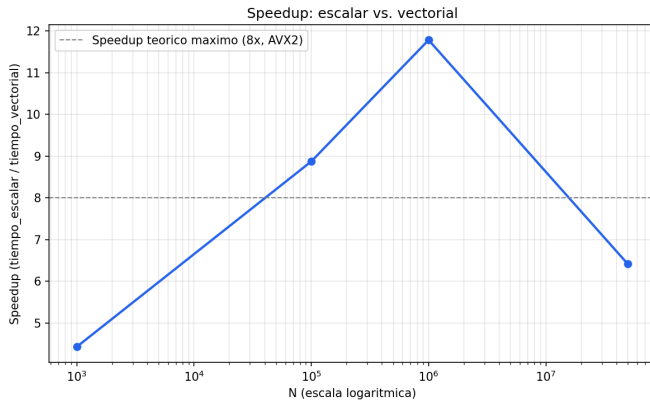


Figura 1. Speedup de la versión vectorial respecto a la escalar en función de N (escala logarítmica). La línea punteada marca el factor teórico de 8x de AVX2.

Con $N = 10^3$ la desviación de la versión vectorial supera a su promedio: el kernel tarda apenas $1,5 \mu\text{s}$, del mismo

orden que la resolución y el costo de las propias llamadas de medición, por lo que ese punto es el menos confiable de la serie.

IX-C. Contadores de hardware

Se ejecutó `perf stat` sobre ambas versiones con el arreglo de 5×10^7 elementos, registrando ciclos, instrucciones, referencias y fallos de caché. Como `perf` mide el proceso completo, incluida la lectura de 200 MB de entrada y la reserva de memoria, cada versión se ejecutó con 1 y con 31 repeticiones del kernel. La diferencia entre ambas corridas corresponde a 30 ejecuciones del kernel aislado, porque la entrada y salida es idéntica en las dos.

```

1 EV=cycles,instructions,cache-references,cache-
  misses
2 perf stat -e $EV ./bin/norm_scalar data/
  input_perf_n50000000.dat /tmp/o.dat 1
3 perf stat -e $EV ./bin/norm_scalar data/
  input_perf_n50000000.dat /tmp/o.dat 31

```

En la corrida bajo `perf`, el tiempo por repetición fue de 302,6 ms para la versión escalar y 65,7 ms para la vectorial, un speedup de 4,6x, menor que el 6,4x de la tabla VI para el mismo tamaño. Ambas corridas se hicieron en el mismo equipo, pero en momentos distintos; en un portátil, el modo de energía y la frecuencia del procesador cambian entre ejecuciones. La conclusión cualitativa, un speedup claramente inferior al obtenido mientras los datos caben en caché, se mantiene en ambas.

En el procesador AMD, el evento genérico `cache-misses` de `perf` contabiliza fallos en el último nivel de caché, es decir, accesos que deben resolverse en memoria principal.

IX-D. Precisión numérica en los tamaños grandes

Al verificar los tamaños de rendimiento con `make verify-scalar` y `make verify-vector`, que comparan también los archivos `input_perf_*` contra la referencia, la varianza se aleja de la referencia conforme crece N (tabla VIII).

Con la tolerancia de 10^{-4} , la versión escalar falla a partir de $N = 10^6$ y la vectorial en $N = 5 \times 10^7$. El error de la varianza se propaga a la desviación y, a través de ella, a todo el arreglo normalizado: en 5×10^7 quedan fuera de tolerancia el 99,9 % de los elementos en la versión escalar y el 97,8 % en la vectorial, y en la escalar fallan además la suma y la

Cuadro VI
TIEMPO Y CICLOS DEL KERNEL (PROMEDIO $\pm$ DESVIACIÓN, 30 REPETICIONES), CICLOS POR ELEMENTO Y SPEEDUP.

N	Escalar			Vectorial			S
	Tiempo (ms)	Ciclos	Ciclos/elem.	Tiempo (ms)	Ciclos	Ciclos/elem.	
10^3	$0,006897 \pm 0,003108$	13 638	13,6	$0,001556 \pm 0,004490$	2 964	2,96	4,43
10^5	$0,6313 \pm 0,0121$	1 260 012	12,6	$0,07115 \pm 0,01221$	141 899	1,42	8,87
10^6	$6,350 \pm 0,088$	12 675 228	12,7	$0,5390 \pm 0,0329$	1 075 834	1,08	11,78
5×10^7	$310,8 \pm 14,4$	620 367 578	12,4	$48,44 \pm 1,31$	96 707 771	1,93	6,41

Cuadro VII
CONTADORES DE PERF DEL KERNEL AISLADO ($N = 5 \times 10^7$,
DIFERENCIA 31 – 1 REPETICIONES, AMD RYZEN 7 7730U).

	Escalar	Vectorial	Relación
Instrucciones (10^6)	48 248	5 646	8,5x menos
Ciclos (10^6)	20 950	4 157	5,0x menos
IPC	2,30	1,36	–
Fallos / referencias	1,77 %	1,91 %	$\approx$ igual

Cuadro VIII
VARIANZA CALCULADA POR CADA VERSIÓN FRENTE A LA REFERENCIA
EN DOBLE PRECISIÓN, Y ERROR RELATIVO.

N	Referencia	Escalar	Vectorial
10^3	3394,8638	3394,8647 ($2,8 \times 10^{-7}$)	3394,8640 ($6,5 \times 10^{-8}$)
10^5	3329,0001	3328,9399 ($1,8 \times 10^{-5}$)	3328,9983 ($5,3 \times 10^{-7}$)
10^6	3332,2132	3330,9983 ($3,6 \times 10^{-4}$)	3332,1519 ($1,8 \times 10^{-5}$)
5×10^7	3333,3822	2914,1631 ($1,3 \times 10^{-1}$)	3314,9788 ($5,5 \times 10^{-3}$)

media (error relativo de $1,7 \times 10^{-4}$). Los mínimos y máximos son exactos en todos los tamaños, porque se obtienen por comparación y no por acumulación. La sección X-E explica la causa.

X. ANÁLISIS Y DISCUSIÓN

X-A. Aceleración teórica frente a la medida

La columna de ciclos por elemento de la tabla VI resume el comportamiento de ambas versiones. La escalar gasta prácticamente lo mismo en todos los tamaños, entre 12,4 y 12,7 ciclos por elemento desde 10^5 hasta 5×10^7 : su velocidad no depende de dónde estén los datos, porque está limitada por su propia secuencia de instrucciones. La vectorial, en cambio, baja hasta 1,08 ciclos por elemento con $N = 10^6$ y sube a 1,93 con 5×10^7 . El speedup es el cociente de ambas columnas, de modo que su forma la determina por completo la versión vectorial.

Con $N = 10^5$ y $N = 10^6$ el speedup supera el factor 8, hasta 11,8x. Superar 8x es posible porque ese factor solo compara el ancho de los registros, mientras que las dos versiones no ejecutan el mismo algoritmo instrucción por instrucción. La versión escalar paga por cada elemento el control del bucle (cmp, jge, inc, jmp) y, en compute_stats, dos comparaciones

con salto para el mínimo y el máximo. Además, sus bucles de suma encadenan cada addss con el anterior a través de un único acumulador, de modo que cada suma debe esperar la latencia de la previa. La vectorial paga el control una vez cada ocho elementos, obtiene mínimo y máximo con vminps/vmaxps sin saltos y avanza ocho elementos por cada eslabón de la cadena de dependencias. El resultado es que ejecuta 8,5 veces menos instrucciones (tabla VII), algo más que el factor 8 del ancho vectorial.

Entre 10^5 y 10^6 la versión vectorial pasa de 1,42 a 1,08 ciclos por elemento. Con un conjunto de trabajo mayor cabría esperar lo contrario, por lo que la diferencia se atribuye a la variabilidad de la medición: el punto de 10^5 dura apenas 71 μ s y tiene una dispersión del 17 %, y en ese intervalo la frecuencia del núcleo puede no haber alcanzado su valor de régimen. En ambos tamaños el resultado es el mismo: mientras los datos residen en caché, el speedup se ubica por encima de 8x.

Con $N = 10^3$ el speedup baja a 4,4x. A este tamaño, la parte fija de cada repetición (las llamadas a las funciones, las reducciones horizontales, los cálculos escalares de la media y la desviación, y las propias llamadas de medición) es comparable al trabajo del bucle, y esa parte no se acelera al vectorizar.

Con $N = 5 \times 10^7$ el speedup cae a 6,4x. Esta caída no se debe a que la versión vectorial ejecute más trabajo, sino a que el tiempo deja de estar determinado por la cantidad de instrucciones, como se discute en la sección X-D.

X-B. Costo de la reducción horizontal

La reducción de la suma requiere cuatro instrucciones y la de cada extremo seis, y se ejecutan una sola vez por llamada. Frente a las $N/8$ iteraciones del bucle principal, su costo es constante y se vuelve despreciable con rapidez: con $N = 10^3$ el bucle ejecuta 125 iteraciones por recorrido y la reducción representa del orden del 1 % de las instrucciones. Su impacto solo sería apreciable para tamaños muy pequeños, en los que el tiempo total es de todas formas insignificante.

X-C. Impacto del remanente

El bucle de remanente procesa como máximo siete elementos por recorrido. Para los tamaños de rendimiento, todos múltiplos de 8, no se ejecuta ninguna iteración de remanente; aun si N no fuera múltiplo de 8, siete elementos escalares frente a millones vectoriales no tienen efecto medible. El remanente solo domina cuando $N < 8$, caso en que la

versión vectorial se comporta como la escalar. En términos de la ley de Amdahl, la fracción no vectorizada dentro de los kernels (remanente, reducciones y cálculo de la media y la raíz cuadrada) es prácticamente nula para arreglos grandes; solo pesa con $N = 10^3$, donde explica el speedup de 4,4x. Para arreglos grandes, la fracción que limita el speedup proviene del acceso a memoria.

X-D. Limitaciones por ancho de banda de memoria

Cada repetición recorre la entrada cuatro veces y escribe la salida una vez, con un conjunto de trabajo de $8N$ bytes. Hasta $N = 10^6$ (8 MB) los datos caben en la L3 de 16 MiB y la reducción de instrucciones se traduce íntegramente en speedup. Con 5×10^7 (400 MB) deben venir de la DRAM: la versión vectorial pasa de 1,08 a 1,93 ciclos por elemento y el speedup cae de 11,8x a 6,4x, mientras que la escalar no cambia, porque es tan lenta que la memoria alcanza a entregarle los datos. Con una o dos operaciones por cada float leído, el kernel tiene una intensidad aritmética muy baja y queda limitado por el ancho de banda, no por el cómputo.

Los contadores de la tabla VII lo confirman: la versión vectorial ejecuta 8,5 veces menos instrucciones pero solo 5 veces menos ciclos, y su IPC cae de 2,30 a 1,36, señal de que el núcleo espera datos. La tasa de fallos es baja y similar en ambas versiones porque el acceso secuencial permite al *prefetcher* anticipar las líneas; la diferencia es que la vectorial consume los datos más rápido. Con 1 GB de tráfico por repetición, sostiene unos 21 GB/s, cerca del ancho de banda práctico de la memoria, frente a unos 3 GB/s de la escalar.

X-E. Pérdida de precisión en la acumulación

Los errores de la tabla VIII también aumentan con N , pero su causa es distinta de la caída del speedup y no tiene relación con la caché: el tamaño de la caché afecta cuánto tarda en llegar un dato, no su valor. El mismo cálculo, reproducido en Python acumulando en float32 en el mismo orden que los kernels, produce exactamente los mismos resultados que el ensamblador, sin que intervenga ninguna jerarquía de memoria. El error es determinista y proviene de la aritmética de punto flotante.

La causa es la absorción de sumandos pequeños en un acumulador grande. Un float32 tiene 24 bits de mantisa, de modo que la separación entre dos valores consecutivos (ulp) crece con la magnitud. En la versión escalar con $N = 5 \times 10^7$, la suma de cuadrados alcanza $\approx 1,7 \times 10^{11}$, magnitud en la que el ulp vale $2^{14} = 16384$. Cada término $(x_i - \mu)^2$ es en promedio ≈ 3333 , menor que medio ulp: al sumarlo, el resultado se redondea al múltiplo de 16384 más cercano y la mayor parte del término se pierde. Por eso la varianza escalar queda 12,6 % por debajo de la referencia. En general, el error de una suma recursiva crece proporcionalmente al número de términos acumulados en un mismo registro.

La versión vectorial distribuye la suma en ocho acumuladores independientes, uno por carril, cada uno con $N/8$ términos. Cada acumulador crece ocho veces menos, su ulp es ocho veces menor y la absorción se atenúa: el error baja

de $1,3 \times 10^{-1}$ a $5,5 \times 10^{-3}$. La vectorización mejora así la precisión como efecto secundario, aunque no lo suficiente para cumplir la tolerancia en el tamaño mayor. Esto muestra además que la observación del enunciado, según la cual el distinto orden de las sumas solo produce diferencias de redondeo, se cumple para tamaños moderados pero no para N del orden de 10^7 .

Corregir el problema sin abandonar float32 requiere cambiar el orden de acumulación: sumar por bloques (acumular un bloque de, por ejemplo, 1024 elementos en un parcial que se reinicia y sumar luego los parciales), usar suma por pares o aplicar la suma compensada de Kahan. Las tres técnicas mantienen los acumuladores pequeños o recuperan el error de redondeo, a cambio de algunas instrucciones adicionales por elemento.

X-F. Limitaciones del trabajo

- La acumulación en float32 no cumple la tolerancia de 10^{-4} para $N \geq 10^6$ en la versión escalar ni para $N = 5 \times 10^7$ en la vectorial (sección X-E).
- El caso `input_extreme` desborda float32 en el cálculo de la varianza, una limitación inherente al formato.
- Las mediciones presentan variabilidad, en especial para los tamaños pequeños, y difieren entre corridas en momentos distintos (el speedup con 5×10^7 fue de 6,4x en la tabla VI y de 4,6x bajo perf); un número mayor de repeticiones, fijar la frecuencia del procesador y asignar el proceso a un núcleo fijo reducirían la dispersión.
- El kernel recorre el arreglo cuatro veces. Fusionar `sum_array` con la primera pasada de `compute_stats` reduciría el tráfico de memoria, que es el factor limitante para arreglos grandes.

XI. CONCLUSIONES

Se implementaron en NASM dos versiones funcionalmente equivalentes de un kernel de normalización z-score: una escalar y una vectorial con AVX2 que procesa ocho elementos por iteración, con manejo explícito del remanente, reducciones horizontales y casos borde. Ambas producen resultados dentro de la tolerancia en todos los casos de verificación, salvo el de valores extremos, que excede el rango de float32.

La versión vectorial ejecuta 8,5 veces menos instrucciones y alcanza un speedup de hasta 11,8x mientras los datos caben en caché, por encima del factor 8 porque elimina además buena parte del control de bucle y de los saltos de la versión escalar. Cuando el arreglo debe leerse desde memoria principal, el speedup cae a 6,4x porque el kernel, de baja intensidad aritmética, queda limitado por el ancho de banda de memoria; la caída del IPC observada con perf confirma ese diagnóstico. Esto explica por qué el speedup real rara vez alcanza el factor teórico de AVX2: vectorizar reduce el cómputo, pero no la cantidad de bytes que deben moverse.

Por último, la evaluación con arreglos grandes reveló que la acumulación secuencial en float32 pierde precisión por absorción, un problema numérico independiente del rendimiento. La versión vectorial lo atenúa al repartir la suma en ocho carriles.

Como trabajo futuro se propone implementar una acumulación por bloques o compensada para cumplir la tolerancia en todos los tamaños, y fusionar recorridos para reducir el tráfico de memoria.

APÉNDICE A CÓDIGO FUENTE

El código fuente completo se entrega como archivo adjunto a este informe. Incluye los kernels en ensamblador (asm/scalar/stats_scalar.asm y asm/vector/statsvar_vector.asm), el *driver* en C (src/driver.c e include/stats.h), el Makefile y los scripts de tools/: generación de datos de prueba, verificación contra la referencia, comparación entre kernels, medición de rendimiento y graficación del speedup.

APÉNDICE B REGISTROS COMPLETOS DE LAS SESIONES DE GDB

Las transcripciones completas de las sesiones de GDB escalar y vectorial, de las cuales se extrajeron los fragmentos de las secciones V y VII, se entregan como archivos adjuntos a este informe.